

Human-Centered Explainability in AI-Assisted Software Development: Design Principles for Trustworthy Developer Tools

Research Paper — Conceptual Review and Design Framework | 2026

Abstract

AI-assisted programming tools can generate code, explain errors, propose tests, and transform natural-language requirements into implementation suggestions. Their usefulness depends not only on generated code quality but also on whether developers can understand, verify, and appropriately challenge the system's suggestions. This paper presents a conceptual review of explainability requirements for AI-assisted software development. It distinguishes between explanation of the generated artifact, explanation of the model's uncertainty, provenance of supporting information, and explanation of tool actions. The paper proposes a human-centered framework built around four principles: traceability, uncertainty communication, actionable verification, and user control. Rather than treating explainability as a single interface feature, the paper frames it as a workflow property that should appear before, during, and after AI-generated changes. The proposed framework can guide future empirical studies evaluating developer trust, verification behavior, cognitive load, and software quality.

1. Introduction

AI coding assistants have become part of modern software-development workflows. Developers can ask a system to generate functions, refactor existing code, explain compiler errors, or propose tests. These capabilities can reduce the amount of routine typing, but they also introduce a new verification problem: the developer must determine whether an automatically generated change is appropriate for the actual system.

Traditional software tools expose many explicit artifacts such as compiler diagnostics, stack traces, test failures, and version-control diffs. AI systems can instead produce plausible natural-language explanations and code that appears coherent even when an assumption is incorrect. This makes explainability relevant not only for machine-learning researchers but also for everyday software engineering.

2. Research Questions

2.1 Main research question

How can explainability be designed into AI-assisted software-development workflows so that developers can verify generated suggestions without surrendering control of the development process?

2.2 Sub-questions

The paper examines: (1) what information developers need to evaluate AI-generated code; (2) how uncertainty and provenance can be communicated; and (3) which interaction patterns can support verification while preserving developer autonomy.

3. Methodology

This is a conceptual literature-based paper. It combines established work on explainable artificial intelligence, human-AI interaction, automation, and software engineering into a design-oriented framework. The paper does not report an original user study, and therefore does not make causal claims about how particular interfaces affect developers.

The framework is derived by identifying recurring requirements in the literature and translating them into software-development-specific design principles. The resulting principles are intended as hypotheses for future empirical testing.

4. Explainability in Software Development

4.1 Artifact-level explanations

An artifact-level explanation describes what the AI changed. For source code, this can include a structured diff, affected files, changed functions, and a concise description of behavioral impact. Showing the exact modification is often more actionable for developers than a generic statement that the code is 'optimized' or 'improved'.

4.2 Rationale and assumptions

Developers may also need to know why a suggestion was generated. A useful explanation can identify assumptions such as expected input types, framework conventions, or inferred requirements. The purpose is not to expose inaccessible internal model reasoning, but to communicate verifiable assumptions that a developer can inspect.

4.3 Uncertainty

AI-generated code can be syntactically valid while being semantically inappropriate. An interface that communicates uncertainty should therefore distinguish between facts directly established by tools and suggestions inferred by the model. For example, a passing unit test is different from a model's statement that the implementation 'should be safe'.

4.4 Provenance

Provenance concerns where supporting information came from. In software development, relevant provenance can include repository files, API documentation, issue descriptions, test output, compiler messages, and configuration. Provenance allows developers to verify whether the AI used information that is current and relevant to the task.

5. Human-Centered Design Framework

5.1 Principle 1 — Traceability

Every significant AI-generated change should be traceable to a concrete artifact. The developer should be able to move from an explanation to the relevant diff, file, test, requirement, or tool output. Traceability reduces the gap between a conversational explanation and the actual software system.

5.2 Principle 2 — Explicit uncertainty

AI suggestions should communicate uncertainty in a way that is specific and actionable. Instead of presenting every output with the same confidence, the system can identify missing context, conflicting evidence, unsupported assumptions, or untested behavior. Uncertainty should not be reduced to a single percentage unless that number has a defensible interpretation.

5.3 Principle 3 — Actionable verification

An explanation becomes more useful when it suggests a concrete verification step. If an AI modifies a database query, the interface might surface relevant tests or recommend reviewing query parameters. If it changes authentication logic, it might identify security-sensitive assumptions and relevant test cases.

5.4 Principle 4 — User control

Developers should remain able to inspect, modify, reject, or revert AI-generated changes. A workflow that automatically applies large changes without clear review boundaries can reduce meaningful human oversight.

User control is therefore both a usability principle and a software-governance mechanism.

6. Proposed Interaction Workflow

A human-centered AI coding workflow can be divided into five stages. First, the developer states a task or requirement. Second, the AI identifies relevant context and presents the proposed change. Third, the system shows a compact explanation with assumptions and provenance. Fourth, the developer verifies the change using diffs, tests, static analysis, or manual inspection. Fifth, the developer accepts, edits, or rejects the result.

This workflow deliberately separates generation from acceptance. The distinction is important because generation is probabilistic while acceptance is an engineering decision. A developer may accept a suggestion even when the AI cannot provide a complete explanation, but the interface should make the remaining uncertainty visible rather than hiding it.

The workflow can also support progressive disclosure. Experienced developers may want a compact diff and test status, while a less familiar user may need additional context about dependencies or framework behavior. Explainability should therefore be layered instead of forcing every user to read a long explanation.

7. Evaluation Framework

Future empirical research could evaluate the framework using controlled programming tasks. Possible measures include defect detection rate, time to verify a change, number of unnecessary accepted changes, frequency of rejected suggestions, and developers' ability to identify unsupported assumptions. Qualitative interviews could examine whether explanations are understandable and whether developers know when additional verification is required.

A particularly important distinction is between perceived trust and calibrated trust. A system may cause users to report high trust while simultaneously increasing acceptance of incorrect code. Therefore, studies should measure behavior and verification accuracy rather than relying only on questionnaires about confidence.

Another relevant factor is task complexity. A simple utility function and a security-sensitive authentication module should not necessarily use the same explanation strategy. Future studies should therefore vary technical risk, codebase familiarity, and amount of available context.

8. Discussion

Explainability for AI-assisted programming differs from explainability in many conventional machine-learning applications because software artifacts are directly inspectable. Developers can often verify a generated change through source code, tests, documentation, and execution. This creates an opportunity to combine model-generated explanations with objective engineering evidence.

At the same time, explanations can become misleading if they are treated as proof. A coherent explanation can describe an incorrect implementation convincingly. For this reason, explainability should complement verification tools rather than replace them. The most useful systems may be those that connect natural-language explanations to concrete evidence and verification actions.

9. Conclusion

AI-assisted software development introduces a new interaction layer between probabilistic generation and deterministic engineering practices. A human-centered explainability approach should help developers understand what changed, which assumptions were made, what evidence supports the suggestion, and how the result can be verified.

The proposed framework—traceability, explicit uncertainty, actionable verification, and user control—provides a conceptual basis for designing and evaluating such tools. Future work should test these principles with developers performing realistic programming tasks and should examine not only productivity, but also

verification quality and the detection of incorrect AI-generated changes.

References

1. Amershi, S., Weld, D., Vorvoreanu, M., et al. (2019). Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. ACM.
2. Doshi-Velez, F., & Kim, B. (2017). Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*.
3. Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. *Artificial Intelligence*, 267, 1–38.
4. Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). 'Why should I trust you?': Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM.
5. Barke, S., James, M. B., & Polikarpova, N. (2023). Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Human-Computer Interaction*, 7(OOPSLA1).
6. GitHub. (2026). *GitHub Copilot Documentation*. Technical documentation on AI-assisted software development workflows and code suggestions.
7. NIST. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. National Institute of Standards and Technology, U.S. Department of Commerce.